

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 7

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement : Implementation of Transfer Learning Using Pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 for Image Classification and Performance Comparison

Theory

Transfer Learning

Transfer learning re-uses a neural network that was already trained on a large dataset (typically ImageNet, 1.2 million images, 1000 classes) as the starting point for a new, usually smaller, task (here, CIFAR-10 with 10 classes).

- **Feature extraction:** The convolutional base of a pretrained model has already learned general-purpose visual features (edges, textures, shapes, object parts). We keep these frozen (non-trainable) and only train a new classifier head on top.
- **Freezing layers:** Setting `base_model.trainable = False` prevents the pretrained weights from being updated during training, which (a) preserves the learned features, (b) drastically reduces the number of trainable parameters, and (c) speeds up training and reduces the chance of overfitting on a small dataset.
- **Fine-tuning:** After the new head has been trained, a *few* of the final convolutional layers of the base model can optionally be unfrozen and trained with a very small learning rate so the pretrained features adapt slightly to the new dataset. Fine-tuning the *entire* network is expensive and often unnecessary — usually only the last block is unfrozen.
- **Why CIFAR-10 needs resizing:** CIFAR-10 images are only $32 \times 32 \times 3$, but VGG16 / ResNet50 / EfficientNetB0 were designed and pretrained on ImageNet images resized to at least $224 \times 224 \times 3$. Feeding 32×32 images directly into these networks would either error out or produce extremely degraded feature maps, so every image is

resized to 224×224 (or a model-appropriate size) inside the `tf.data` pipeline before it reaches the model.

- Why the final layer must be replaced: The original classifier head of these networks was trained to output 1000 ImageNet classes. We remove it (`include_top=False`) and replace it with a small new head ending in a `Dense(10, activation="softmax")` layer suited to CIFAR-10's 10 classes.

1. AlexNet

AlexNet (Krizhevsky et al., 2012) was one of the first deep CNNs to win the ImageNet competition. It uses stacked convolution + ReLU + max-pooling blocks, followed by fully-connected layers with dropout, ending in a softmax classifier. Important: `tf.keras.applications` does not provide an official ImageNet-pretrained AlexNet. In this notebook, AlexNet is therefore implemented manually as an AlexNet-style CNN, scaled down for CIFAR-10, and trained from scratch (random initialization) — it is not transfer learning.

2. VGG16

VGG16 (Simonyan & Zisserman, 2014) uses a very uniform architecture of stacked 3×3 convolutions and 2×2 max-pooling, with 16 weight layers total. It is available in `tf.keras.applications.VGG16` with official ImageNet weights.

3. ResNet50

ResNet50 (He et al., 2015) introduced residual (skip) connections, which allow gradients to flow through very deep networks (50 layers here) without vanishing, enabling much deeper architectures to be trained successfully. Available in `tf.keras.applications.ResNet50` with official ImageNet weights.

4. EfficientNetB0

EfficientNetB0 (Tan & Le, 2019) uses compound scaling (jointly scaling network depth, width, and input resolution) discovered via neural architecture search, achieving strong accuracy with comparatively few parameters.

Algorithm

1. Load CIFAR-10 (`tf.keras.datasets.cifar10.load_data()`).
2. Take a configurable subset (`TRAIN_SAMPLES`, `TEST_SAMPLES`) and split off a validation set.
3. Detect available GPU(s); enable memory growth and mixed precision only if a GPU is found.

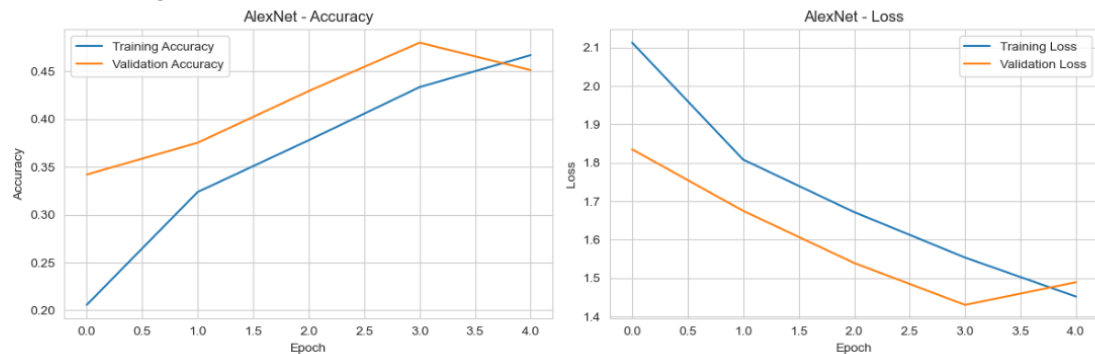
4. For each architecture (AlexNet-style, VGG16, ResNet50, EfficientNetB0):
 - a. Build a tf.data pipeline with model-specific resizing and preprocessing.
 - b. Load/build the architecture — pretrained ImageNet weights where officially available (VGG16, ResNet50, EfficientNetB0), from-scratch random initialization for AlexNet-style.
 - c. Replace/attach a CIFAR-10-specific classification head ending in Dense(10, softmax).
 - d. Freeze the pretrained convolutional base (feature-extraction transfer learning) where applicable.
 - e. Train with early stopping, LR reduction on plateau, and checkpointing.
 - f. Evaluate: test loss/accuracy, precision/recall/F1, confusion matrix, classification report, sample predictions.
 - g. Save the trained model to disk; record its metrics; free GPU memory (del model, gc.collect(), tf.keras.backend.clear_session()) before starting the next architecture.
5. Combine all four models' metrics into a single comparison table and set of charts.
6. Discuss and conclude based only on the actually measured numbers.

Output :

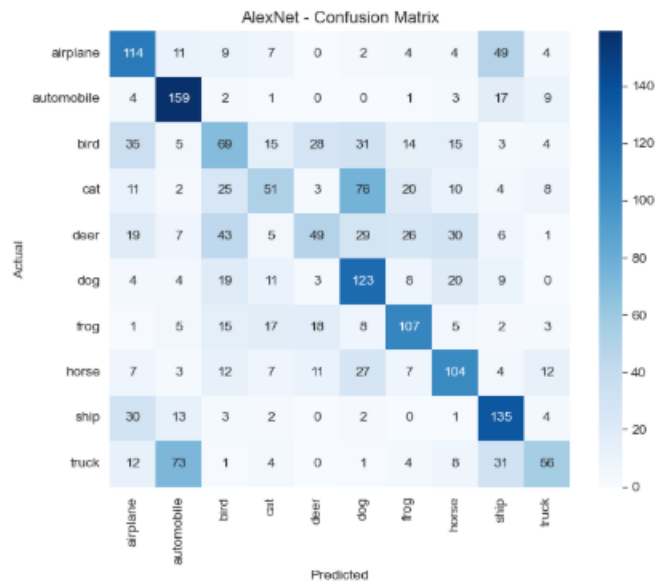
AlexNet Training

```
[49]: alexnet_history, alexnet_time = train_model(alexnet_model, alexnet_train_ds, alexnet_val_ds,  
                                              model_name="alexnet", epochs=EPOCHS)  
plot_history(alexnet_history, "AlexNet")
```

```
--- Training alexnet ---  
Epoch 1/5  
1063/1063 ----- 44s 39ms/step - accuracy: 0.2056 - loss: 2.1126 - val_accuracy: 0.3420 - val_loss: 1.8351 - learning_rate: 1.0000e-04  
Epoch 2/5  
1063/1063 ----- 42s 40ms/step - accuracy: 0.3238 - loss: 1.8085 - val_accuracy: 0.3753 - val_loss: 1.6752 - learning_rate: 1.0000e-04  
Epoch 3/5  
1063/1063 ----- 42s 40ms/step - accuracy: 0.3779 - loss: 1.6719 - val_accuracy: 0.4293 - val_loss: 1.5395 - learning_rate: 1.0000e-04  
Epoch 4/5  
1063/1063 ----- 42s 40ms/step - accuracy: 0.4335 - loss: 1.5535 - val_accuracy: 0.4800 - val_loss: 1.4304 - learning_rate: 1.0000e-04  
Epoch 5/5  
1063/1063 ----- 42s 40ms/step - accuracy: 0.4669 - loss: 1.4520 - val_accuracy: 0.4513 - val_loss: 1.4895 - learning_rate: 1.0000e-04  
--- Finished training alexnet in 212.8 seconds ---
```



```
--- AlexNet Evaluation ---  
Test Loss : 1.4180  
Test Accuracy : 0.4835  
Precision : 0.4786  
Recall : 0.4835  
F1-score : 0.4651  
Training time : 212.8 s  
Total params : 2,547,722  
Trainable params: 2,547,722
```



Classification Report:				
	precision	recall	f1-score	support
airplane	0.48	0.56	0.52	204
automobile	0.56	0.81	0.67	196
bird	0.35	0.32	0.33	219
cat	0.42	0.24	0.31	210
deer	0.44	0.23	0.30	215
dog	0.41	0.61	0.49	201
frog	0.56	0.59	0.58	181
horse	0.52	0.54	0.53	194
ship	0.52	0.71	0.60	190
truck	0.55	0.29	0.38	190
accuracy			0.48	2000
macro avg	0.48	0.49	0.47	2000
weighted avg	0.48	0.48	0.47	2000

AlexNet - Sample Predictions

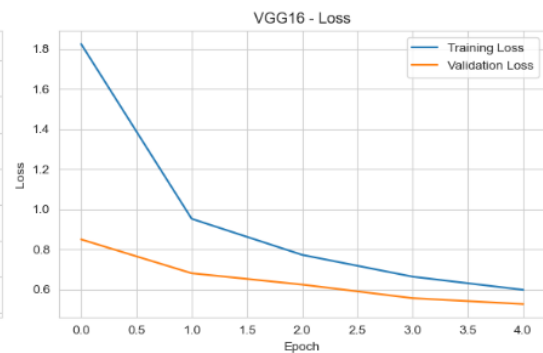
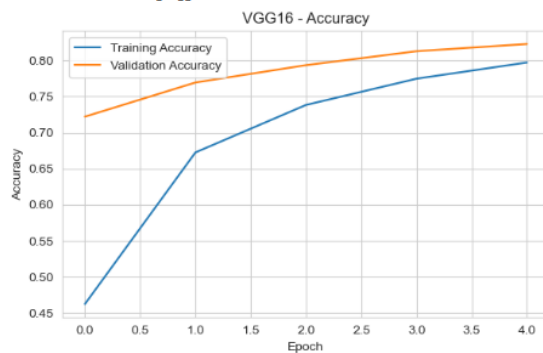


AlexNet model saved.

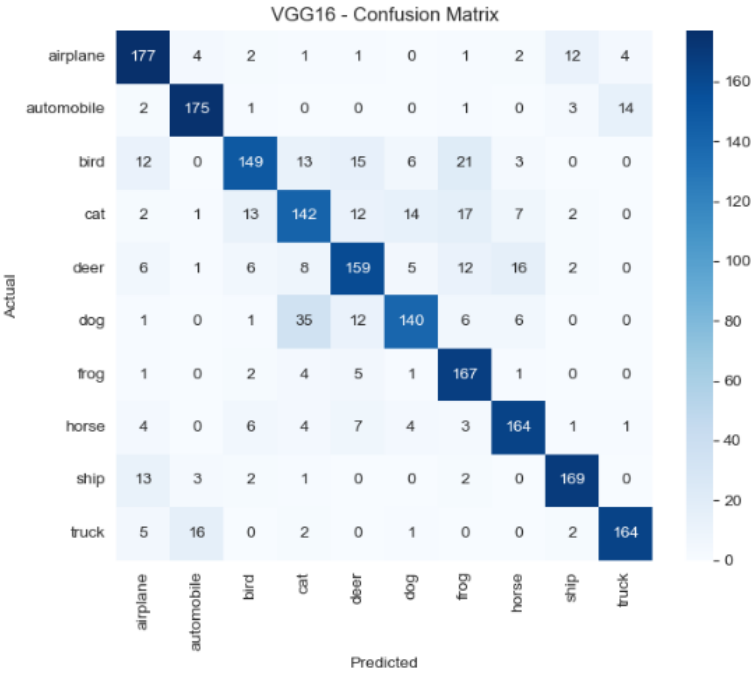
VGG16 Training

```
[29]: vgg16_history, vgg16_time = train_model(vgg16_model, vgg16_train_ds, vgg16_val_ds,
      model_name="vgg16", epochs=EPOCHS)
      plot_history(vgg16_history, "VGG16")

--- Training vgg16 ---
Epoch 1/5 1599s 1s/step - accuracy: 0.4625 - loss: 1.8249 - val_accuracy: 0.7220 - val_loss: 0.8505 - learning_rate: 1.0000e-04
Epoch 2/5 1063/1063 733s 689ms/step - accuracy: 0.6725 - loss: 0.9533 - val_accuracy: 0.7693 - val_loss: 0.6817 - learning_rate: 1.0000e-04
Epoch 3/5 1063/1063 735s 692ms/step - accuracy: 0.7382 - loss: 0.7733 - val_accuracy: 0.7933 - val_loss: 0.6251 - learning_rate: 1.0000e-04
Epoch 4/5 1063/1063 735s 691ms/step - accuracy: 0.7746 - loss: 0.6645 - val_accuracy: 0.8127 - val_loss: 0.5573 - learning_rate: 1.0000e-04
Epoch 5/5 1063/1063 737s 694ms/step - accuracy: 0.7971 - loss: 0.5989 - val_accuracy: 0.8227 - val_loss: 0.5279 - learning_rate: 1.0000e-04
--- Finished training vgg16 in 4539.7 seconds ---
```

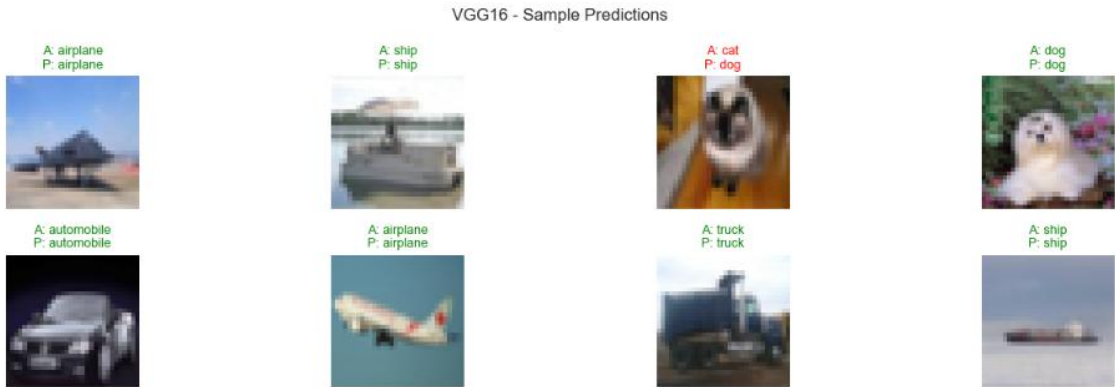


=== VGG16 Evaluation ===
Test Loss : 0.5591
Test Accuracy : 0.8030
Precision : 0.8055
Recall : 0.8030
F1-score : 0.8018
Training time : 4539.7 s
Total params : 14,848,586
Trainable params: 133,898



Classification Report:

	precision	recall	f1-score	support
airplane	0.79	0.87	0.83	204
automobile	0.88	0.89	0.88	196
bird	0.82	0.68	0.74	219
cat	0.68	0.68	0.68	210
deer	0.75	0.74	0.75	215
dog	0.82	0.70	0.75	201
frog	0.73	0.92	0.81	181
horse	0.82	0.85	0.83	194
ship	0.88	0.89	0.89	190
truck	0.90	0.86	0.88	190
accuracy			0.80	2000
macro avg	0.81	0.81	0.80	2000
weighted avg	0.81	0.80	0.80	2000

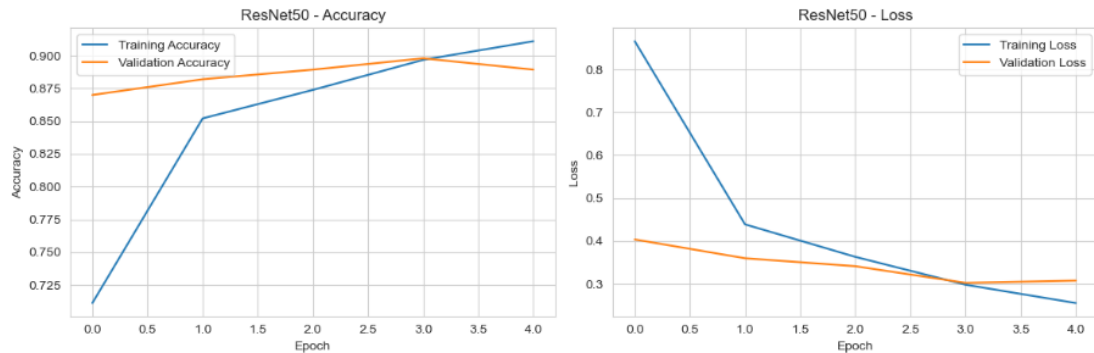


VGG16 model saved.

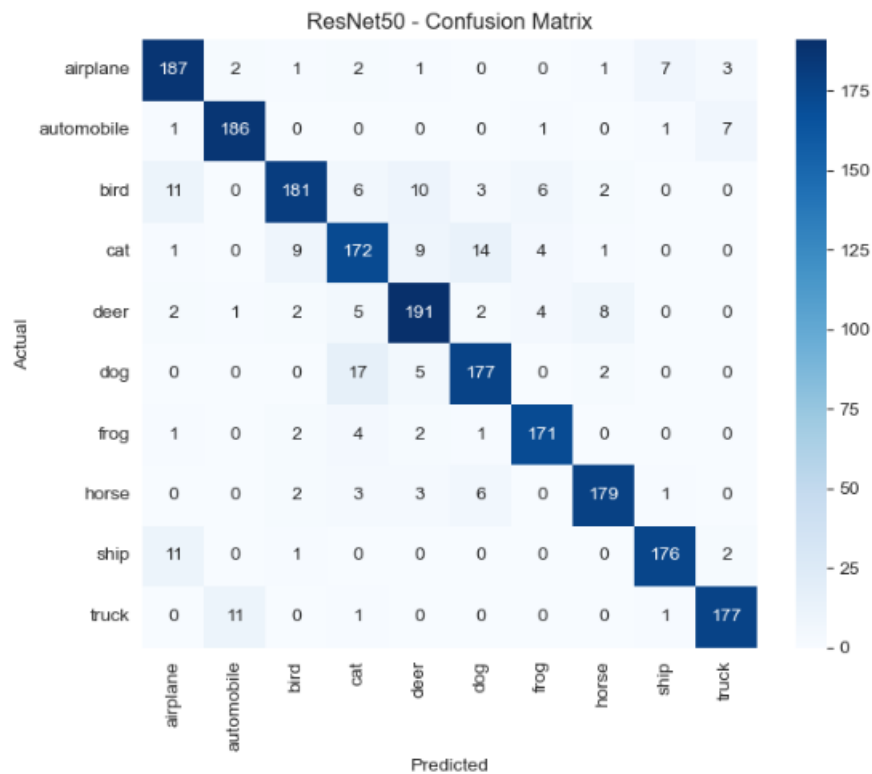
ResNet50 Training

```
[34]: resnet50_history, resnet50_time = train_model(resnet50_model, resnet50_train_ds, resnet50_val_ds,
                                                model_name="resnet50", epochs=EPOCHS)
plot_history(resnet50_history, "ResNet50")
```

```
--- Training resnet50 ---
Epoch 1/5
1063/1063 ----- 426s 395ms/step - accuracy: 0.7112 - loss: 0.8663 - val_accuracy: 0.8700 - val_loss: 0.4039 - learning_rate: 1.0000e-04
Epoch 2/5
1063/1063 ----- 421s 396ms/step - accuracy: 0.8521 - loss: 0.4391 - val_accuracy: 0.8820 - val_loss: 0.3600 - learning_rate: 1.0000e-04
Epoch 3/5
1063/1063 ----- 423s 398ms/step - accuracy: 0.8739 - loss: 0.3633 - val_accuracy: 0.8893 - val_loss: 0.3415 - learning_rate: 1.0000e-04
Epoch 4/5
1063/1063 ----- 427s 402ms/step - accuracy: 0.8968 - loss: 0.2983 - val_accuracy: 0.8980 - val_loss: 0.3026 - learning_rate: 1.0000e-04
Epoch 5/5
1063/1063 ----- 419s 394ms/step - accuracy: 0.9111 - loss: 0.2555 - val_accuracy: 0.8893 - val_loss: 0.3078 - learning_rate: 1.0000e-04
--- Finished training resnet50 in 2115.8 seconds ---
```



```
=== ResNet50 Evaluation ===
Test Loss      : 0.3142
Test Accuracy  : 0.8985
Precision      : 0.8989
Recall         : 0.8985
F1-score       : 0.8984
Training time  : 2115.8 s
Total params   : 24,114,826
Trainable params: 527,114
```



Classification Report:				
	precision	recall	f1-score	support
airplane	0.87	0.92	0.89	204
automobile	0.93	0.95	0.94	196
bird	0.91	0.83	0.87	219
cat	0.82	0.82	0.82	210
deer	0.86	0.89	0.88	215
dog	0.87	0.88	0.88	201
frog	0.92	0.94	0.93	181
horse	0.93	0.92	0.93	194
ship	0.95	0.93	0.94	190
truck	0.94	0.93	0.93	190
accuracy			0.90	2000
macro avg	0.90	0.90	0.90	2000
weighted avg	0.90	0.90	0.90	2000

ResNet50 - Sample Predictions

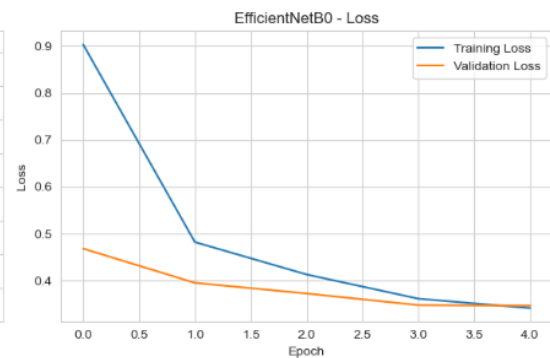
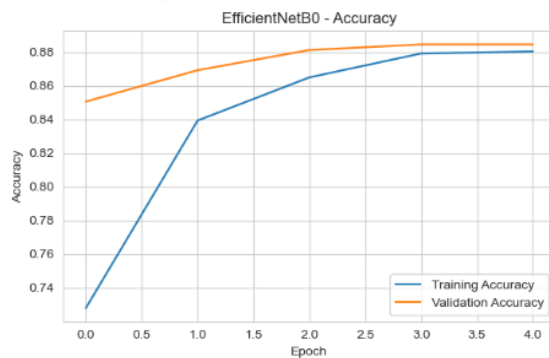


ResNet50 model saved.

EfficientNetB0 Training

```
[39]: efficientnet_history, efficientnet_time = train_model(
    efficientnet_model, efficientnet_train_ds, efficientnet_val_ds,
    model_name="efficientnetb0", epochs=EPOCHS,
)
plot_history(efficientnet_history, "EfficientNetB0")
```

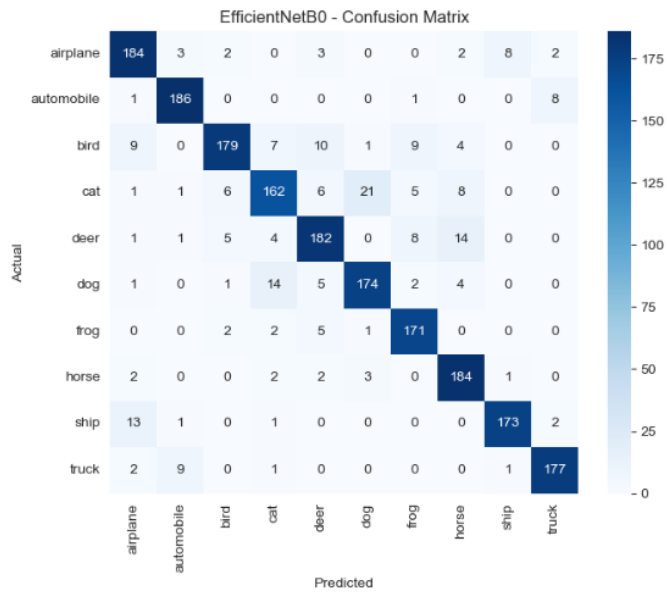
```
--- Training efficientnetb0 ---
Epoch 1/5
1063/1063 ----- 206s 186ms/step - accuracy: 0.7279 - loss: 0.9038 - val_accuracy: 0.8507 - val_loss: 0.4685 - learning_rate: 1.0000e-04
Epoch 2/5
1063/1063 ----- 202s 190ms/step - accuracy: 0.8394 - loss: 0.4823 - val_accuracy: 0.8693 - val_loss: 0.3954 - learning_rate: 1.0000e-04
Epoch 3/5
1063/1063 ----- 202s 190ms/step - accuracy: 0.8651 - loss: 0.4132 - val_accuracy: 0.8813 - val_loss: 0.3727 - learning_rate: 1.0000e-04
Epoch 4/5
1063/1063 ----- 207s 195ms/step - accuracy: 0.8793 - loss: 0.3616 - val_accuracy: 0.8847 - val_loss: 0.3479 - learning_rate: 1.0000e-04
Epoch 5/5
1063/1063 ----- 203s 191ms/step - accuracy: 0.8805 - loss: 0.3415 - val_accuracy: 0.8847 - val_loss: 0.3469 - learning_rate: 1.0000e-04
--- Finished training efficientnetb0 in 1020.3 seconds ---
```



```

=== EfficientNetB0 Evaluation ===
Test Loss      : 0.3419
Test Accuracy  : 0.8860
Precision      : 0.8867
Recall         : 0.8860
F1-score       : 0.8854
Training time  : 1020.3 s
Total params   : 4,380,077
Trainable params: 330,506

```



Classification Report:

	precision	recall	f1-score	support
airplane	0.86	0.90	0.88	204
automobile	0.93	0.95	0.94	196
bird	0.92	0.82	0.86	219
cat	0.84	0.77	0.80	210
deer	0.85	0.85	0.85	215
dog	0.87	0.87	0.87	201
frog	0.87	0.94	0.91	181
horse	0.85	0.95	0.90	194
ship	0.95	0.91	0.93	190
truck	0.94	0.93	0.93	190
accuracy			0.89	2000
macro avg	0.89	0.89	0.89	2000
weighted avg	0.89	0.89	0.89	2000

EfficientNetB0 - Sample Predictions



EfficientNetB0 model saved.

[51]:

	Model	Test Accuracy	Precision	Recall	F1 Score	Training Time (s)	Total Parameters	Trainable Parameters	Train Samples	Test Samples	Epochs	Pretrained (ImageNet)	Fine-tuned
0	EfficientNetB0	0.8860	0.8867	0.8860	0.8854	1020.3	4380077	330506	8500	2000	5	True	False
1	ResNet50	0.8985	0.8989	0.8985	0.8984	2115.8	24114826	527114	8500	2000	5	True	False
2	VGG16	0.8030	0.8055	0.8030	0.8018	4539.7	14848586	133898	8500	2000	5	True	False
3	AlexNet	0.4835	0.4786	0.4835	0.4651	212.8	2547722	2547722	8500	2000	5	False	False

Final Results Table

Saved to `model_comparison.csv` for submission alongside this notebook.

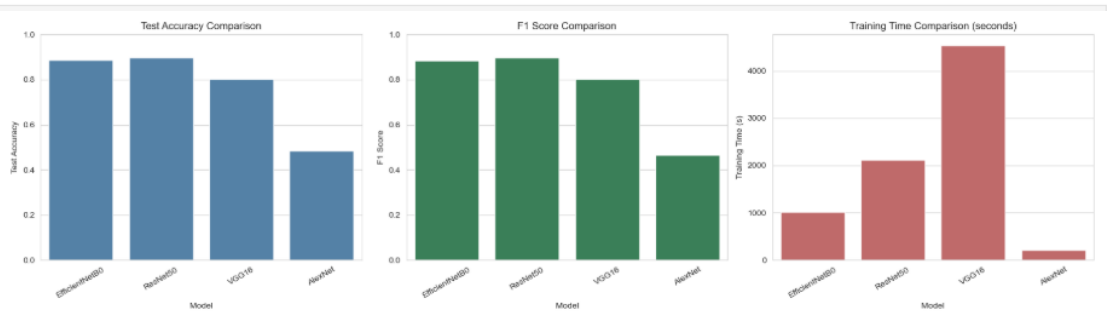
[52]:

```
comparison_df.to_csv("model_comparison.csv", index=False)
print("Saved model_comparison.csv")
comparison_df
```

Saved model_comparison.csv

[53]:

	Model	Test Accuracy	Precision	Recall	F1 Score	Training Time (s)	Total Parameters	Trainable Parameters	Train Samples	Test Samples	Epochs	Pretrained (ImageNet)	Fine-tuned
0	EfficientNetB0	0.8860	0.8867	0.8860	0.8854	1020.3	4380077	330506	8500	2000	5	True	False
1	ResNet50	0.8985	0.8989	0.8985	0.8984	2115.8	24114826	527114	8500	2000	5	True	False
2	VGG16	0.8030	0.8055	0.8030	0.8018	4539.7	14848586	133898	8500	2000	5	True	False
3	AlexNet	0.4835	0.4786	0.4835	0.4651	212.8	2547722	2547722	8500	2000	5	False	False



[54]:

```
# Parameter count comparison (Log scale, since pretrained models have far more params than AlexNet-style CNN)
plt.figure(figsize=(8, 5))
sns.barplot(data=comparison_df, x="Model", y="Total Parameters", color="slateblue")
plt.yscale("log")
plt.title("Total Parameter Count Comparison (log scale)")
plt.ylabel("Total Parameters (log scale)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

